

B0911007H-01/02 2025-2026学年春季学期

计算机组成原理（研讨课）

E02 Git 基础知识

唐睿、陈伟伦
2026年3月12日



中国科学院大学
University of Chinese Academy of Sciences



中科院计算所
INSTITUTE OF COMPUTING TECHNOLOGY, CAS

- ❑ Git 是什么
- ❑ 为什么需要用 Git
- ❑ Git 如何工作
- ❑ Git 的基本用法

- Git 是什么
- 为什么需要用 Git
- Git 如何工作
- Git 的基本用法

Git是什么



- ❑ Git是开源的分布式版本控制系统，用于敏捷高效地管理项目，广泛应用于代码版本管理和协作开发
- ❑ GitHub/Gitee是面向开源和私有软件项目的公共托管平台，支持基于Git版本库格式的代码托管，为开发者提供了代码存储、协作和分享的一站式解决方案
- ❑ GitLab是一个开源的代码仓库管理系统，基于 Git 作为核心代码管理工具，不仅支持代码托管，还集成了项目管理、持续集成/持续交付（CI/CD）等功能
- ❑ 实验课基于开源GitLab为同学们搭建了代码仓库托管服务



- Git 是什么
- 为什么需要用 Git
- Git 如何工作
- Git 的基本用法

为什么需要用 Git：单人工作遇到的难题（1）



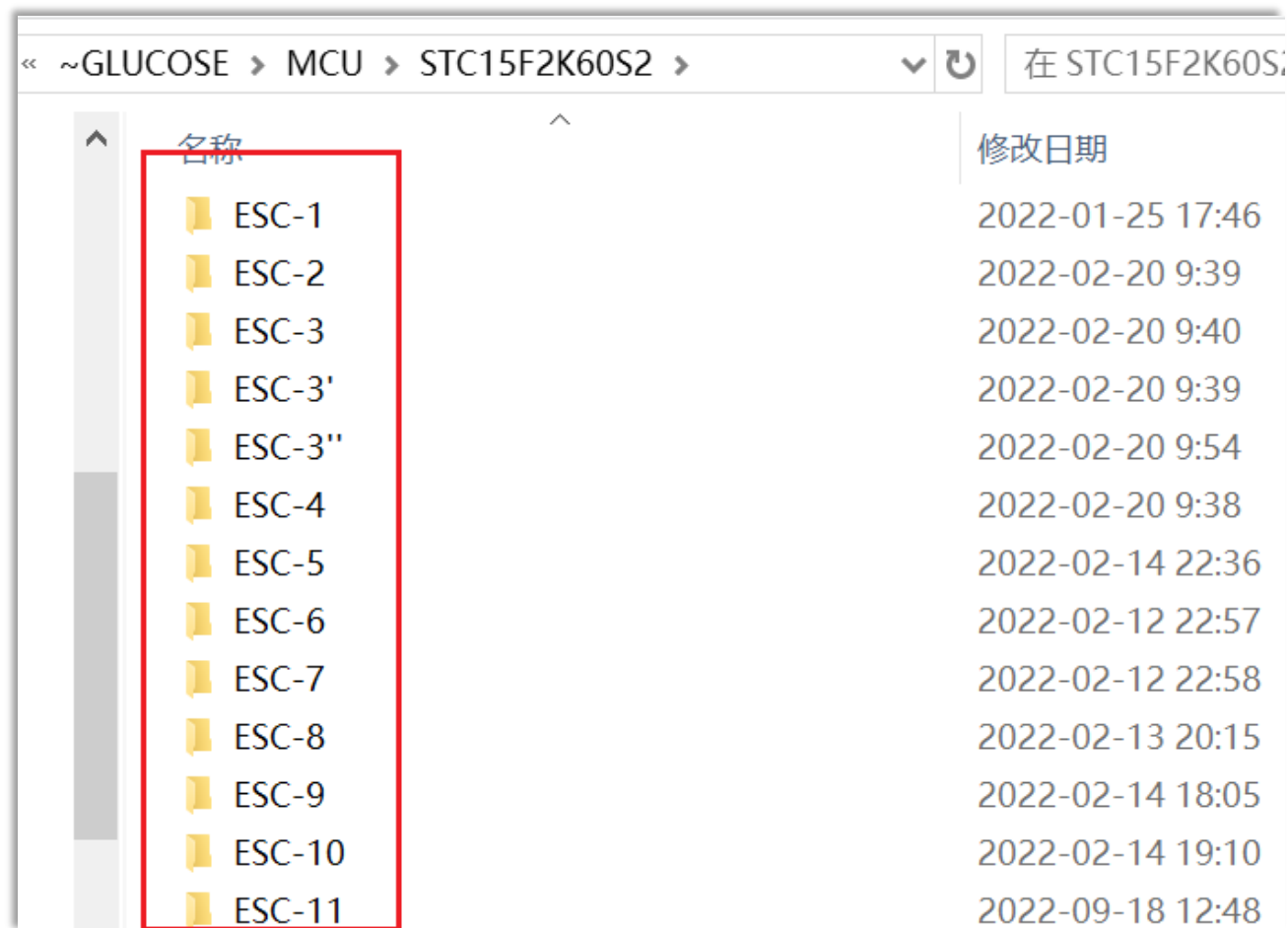
- ❑ 大型项目难以一次性完成，直接修改代码后，旧版本丢失，难以回溯
- ❑ 例：X同学这学期选修了COD-LAB，沿用管理C语言程序设计课代码的方式，学期中发现代码量极大，已记不太清某些设计细节，甚至不确定实现的功能
- ❑ ——早该做记录了！
- ❑ 继续实验时，X同学发现思路有误，但代码已被改得面目全非，想回退却找不到旧版本，只能从头开始。
- ❑ ——应该存有各个版本的代码，方便版本回退！

为什么需要用 Git: 单人工作遇到的难题 (2)



于是只好这样，
每完成一阶段就
复制一份代码

这种方法虽然可行，但代价是
占用大量存储空间，且管理起
来非常繁琐



为什么需要用 Git：多人协作遇到的难题



- ❑ X同学用这种方法熬过了COD-LAB，但接下来面临数据结构大作业——三人小组合作
- ❑ 问题来了：
 - 如何同步三个人对代码的更新？
 - 代码说明文档又该如何记录？
- ❑ 通过QQ或U盘传输？
 - 太麻烦且容易混乱……

- 使用代码托管服务备份文件
- 整合更新内容
- 回溯历史版本
- 对比查看代码的更新细节

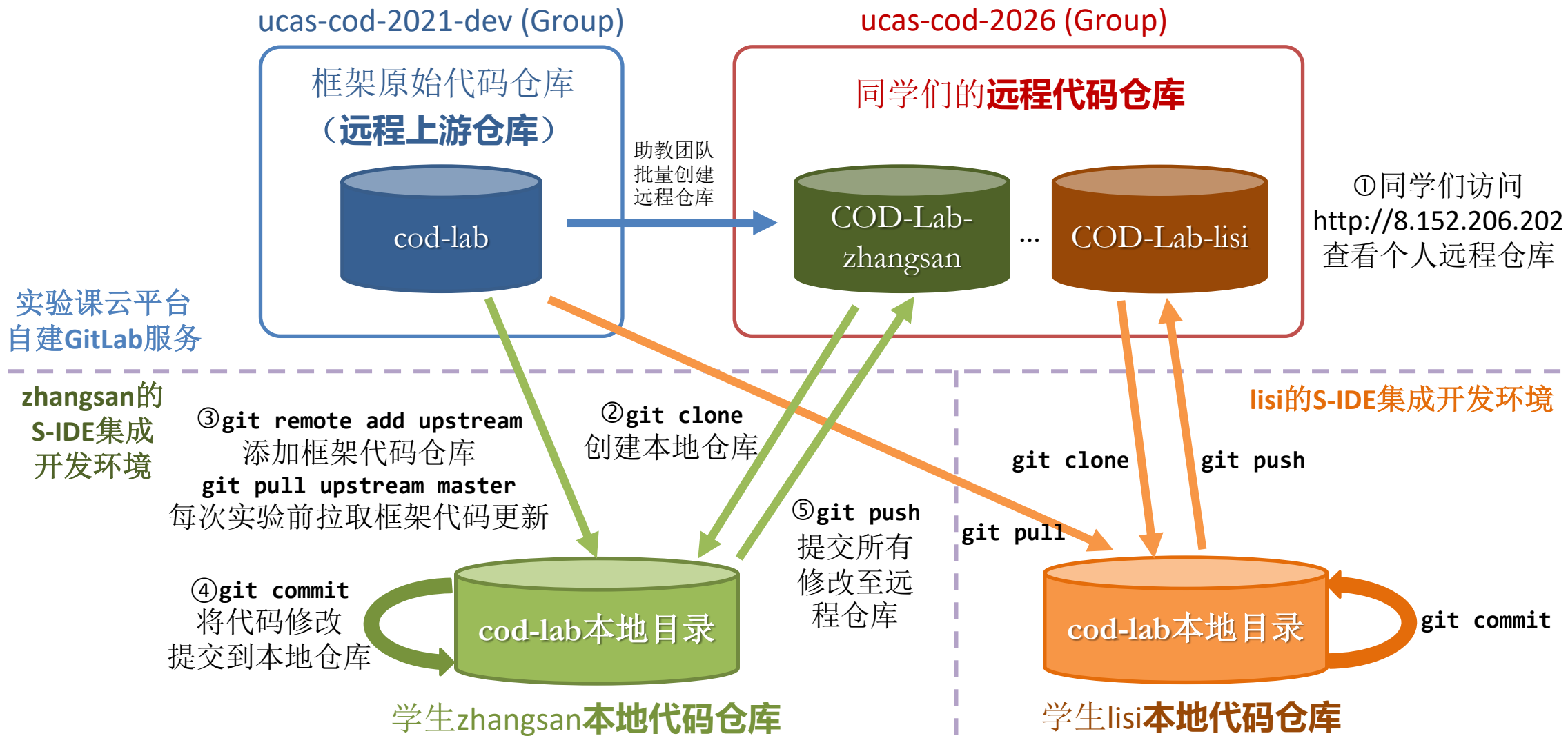
- Git 是什么
- 为什么需要用 Git
- **Git 如何工作**
- Git 的基本用法

- 本地仓库：由.git目录（存储版本信息） 和需要管理的文件组成
- 远程仓库：存储在云服务器（如 GitLab、GitHub、Gitee）上的代码仓库，用于备份和协作
- 远程上游仓库：通常是项目的官方仓库或主仓库

工作原理：

- 本地：在本地记录修改信息，管理不同版本，以实现版本对比、回退及分支管理
- 远程：通过代码托管服务实现代码备份、多人协作及仓库协同管理

实验课中本地与远程仓库的关系及git关键操作

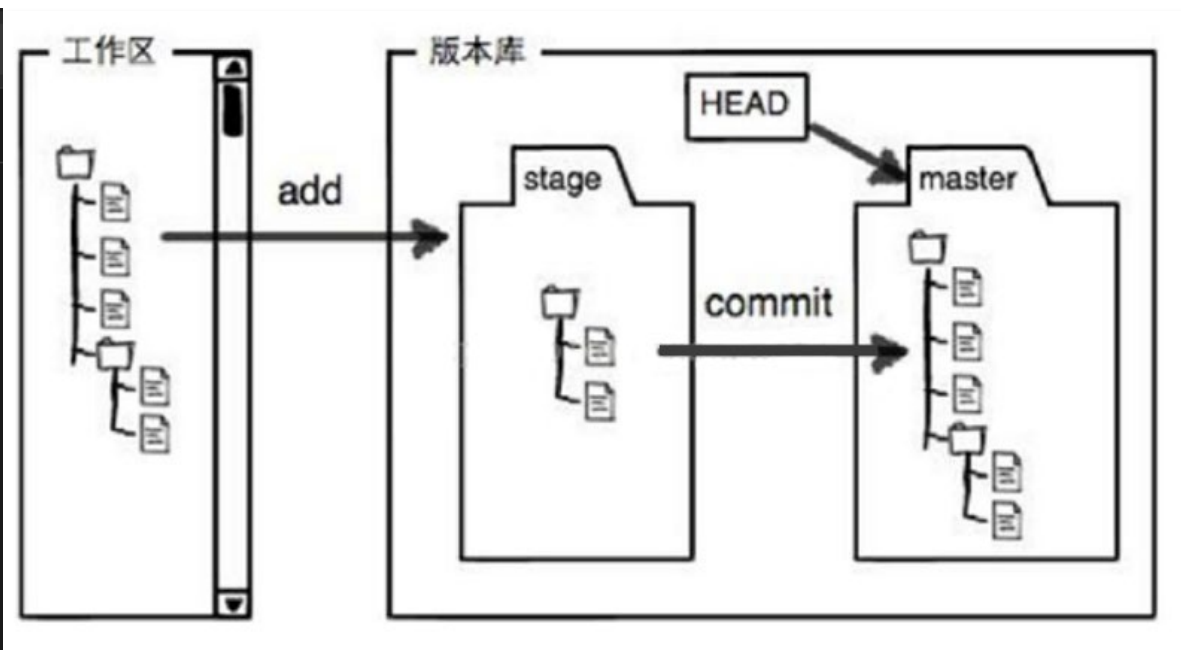
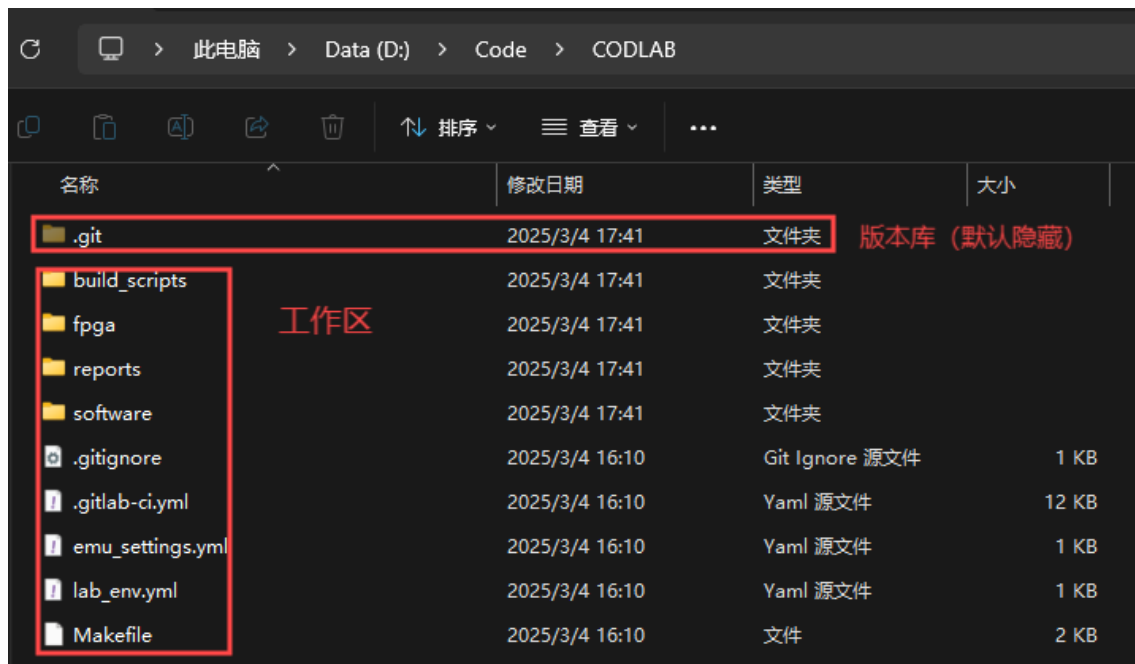


- Git 是什么
- 为什么需要用 Git
- Git 如何工作
- **Git 的基本用法**

Git 用法：基本概念



- ❑ 工作区：当前文件所在目录
- ❑ 暂存区：存储即将提交的文件修改
- ❑ 版本库：存储所有提交的历史记录



- Git 是什么
- 为什么需要用 Git
- Git 如何工作
- **Git 的基本用法**
 - 实验课必要的Git命令

Git 用法：远程仓库推送/拉取

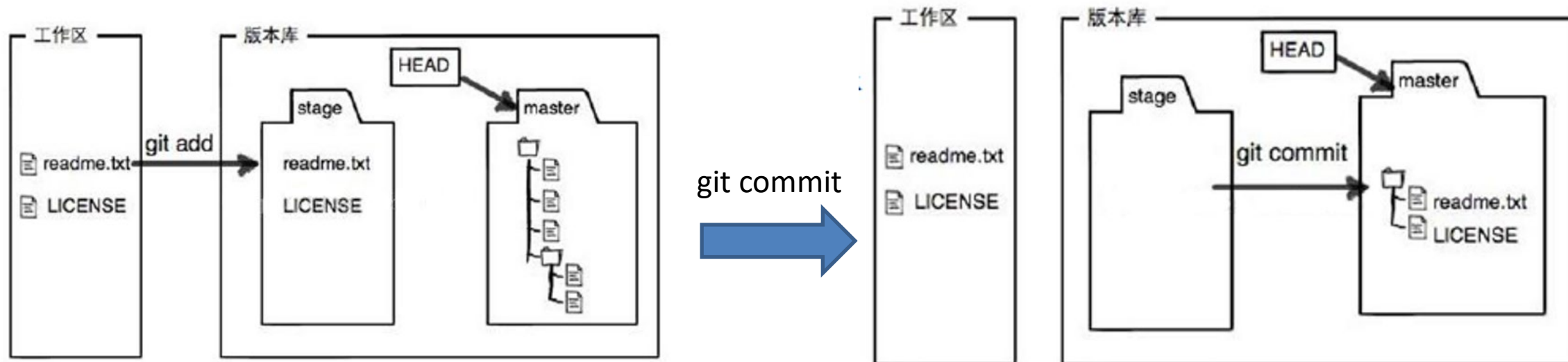


❑ add: 将当前工作区的文件修改添加到暂存区，为提交做准备

- `git add <file1> ...` 或用 `.` 代表所有文件

❑ commit: 将暂存区中的修改确认并提交到当前分支（记为 HEAD），这将记录一个版本（默认会有一个master分支），同时清空暂存区

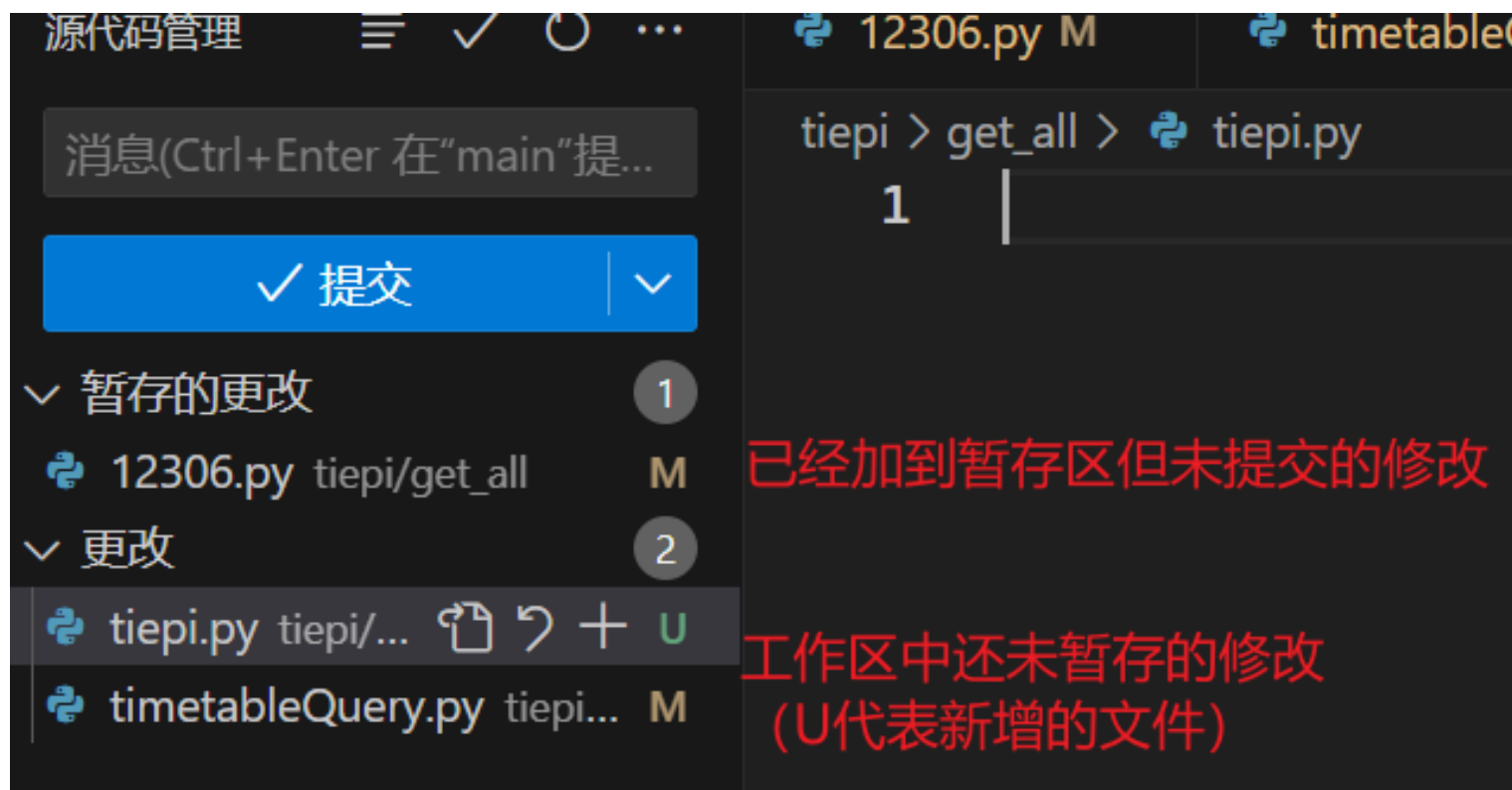
- `git commit -m "<commit message>"` 其中 `<commit message>` 为本次提交的备注信息，用于描述修改内容（引号必须使用）



Git 用法: add 和 commit 示例



□ 下图为在VS Code的插件中直观查看的效果



符号 **U** (Untracked): 未被Git跟踪的文件, 可以进行git add操作

符号 **M** (Modified): 该文件已被修改, 但还未git commit

Commit成功后, 文件名后面的标志符号则会消失

Git 用法：远程仓库推送/拉取



- ❑ remote: 用于添加、查看或删除与远端仓库的连接
 - `git remote add upstream <URL>` 添加一个上游仓库并标记为 upstream
 - `git remote -v` 查看远程仓库
 - `git remote remove <name>` 删除远程仓库
- ❑ push: 将本地仓库的提交推送到远程仓库，完成远程仓库的更新
- ❑ pull: 拉取远程仓库最新文件

```
git add wuziqi.c cod.c //将修改过的wuziqi.c和cod.c加入暂存区
git commit -m "add new codes of wuziqi.c and cod.c" //将暂存区修改信息提交，-m参数后面双引号里面是修改信息
git push origin master //将暂存区的修改内容提交到远程仓库，完成更新
```

开始每个项目前 **git pull upstream master** 获取最新实验框架

Git 用法：标签 (tag)



- ❑ 给仓库历史中的某个commit打上标签 (tag) ，便于标记重要版本（如发布节点）
 - **git tag** //列出标签
 - **git tag -a <tag-name> [commit hash] -m <message>** //基于当前commit或某一确定commit创建标签
 - **git show <tag-name>** //查看该标签及其对应的commit信息
- ❑ 本课程会用到，用来标记某个实验项目中多个阶段的最终文件版本，后续课件会给出具体操作

- Git 是什么
- 为什么需要用 Git
- Git 如何工作
- **Git 的基本用法**
 - 实验课必要的Git命令
 - 开发过程中可能会用到的其他Git命令

Git 用法：查看状态命令



使用下面的命令来查看状态：

- ❑ `git log`: 查看当前分支的提交记录
- ❑ `git status`: 查看提交的状态，查看是否有修改
- ❑ `git diff`: 查看**尚未暂存**文件的具体修改内容

Git 用法：对本地仓库中的修改不满意，要回退（1）

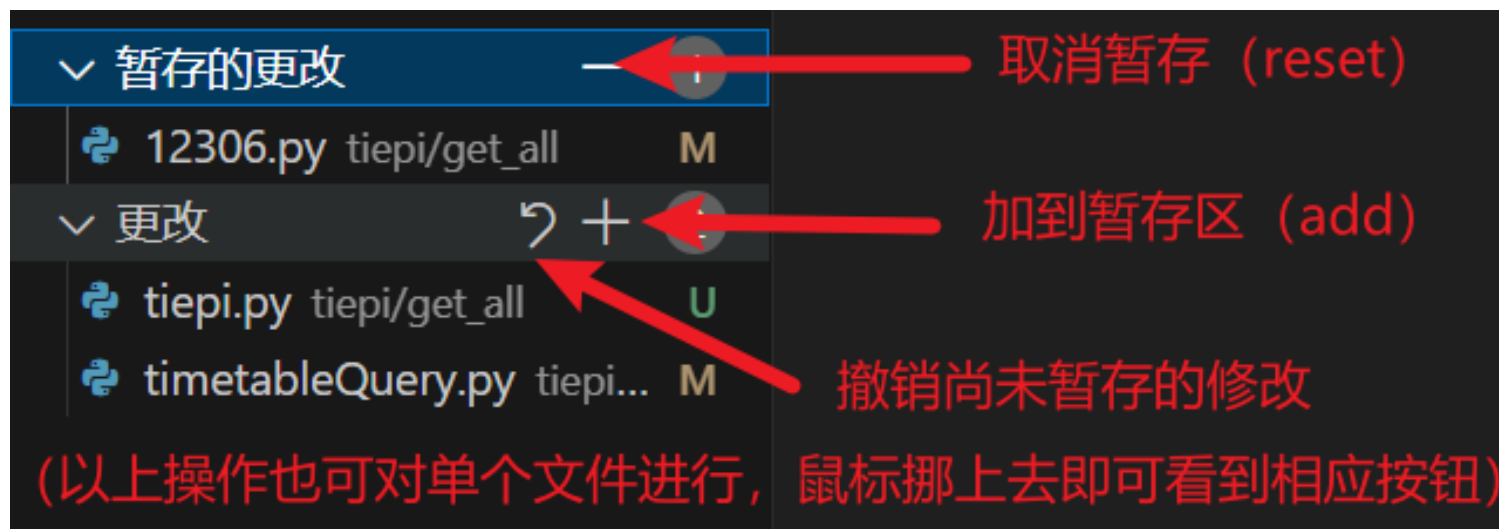
❑ 若修改还没有加到暂存区：

- 用 `git checkout -- <file1> <dir1>` 撤销修改

❑ 若修改已经暂存但还没有提交（commit）：

- 用 `git reset HEAD <file>` 即可转为上一种情况

❑ 这些操作可以在VS Code插件中快捷操作



Git 用法：对本地仓库中的修改不满意，要回退（2）

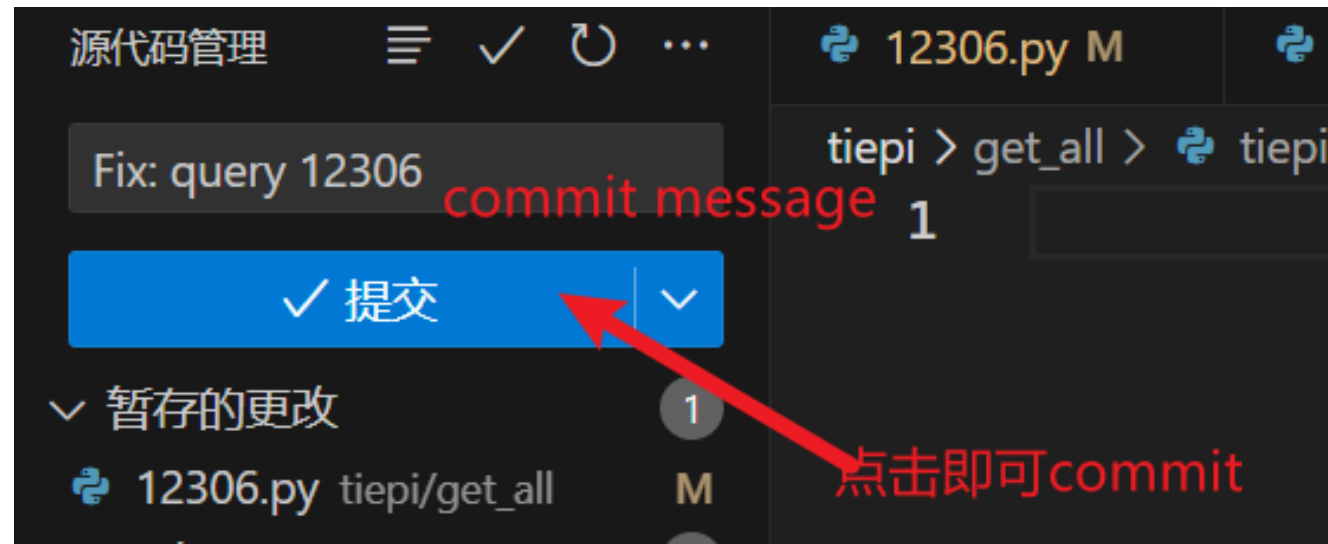
❑ 若修改还没有加到暂存区：

- 用 `git checkout -- <file1> <dir1>` 撤销修改

❑ 若修改已经暂存但还没有提交（commit）：

- 用 `git reset HEAD <file>` 即可转为上一种情况

❑ 这些操作可以在VS Code插件中快捷操作



Git 用法：对本地仓库中的修改不满意，要回退（3）

□ 若修改已经提交

- 用 `git reset <commit hash>` 即可回退到指定提交版本（commit hash 理解为提交版本的ID）

```
PS D:\Code\COD_LAB> git log
commit 7efc670e0290753713ab9728d6831a5ccb240c93 (HEAD -> main, origin/main, origin/HEAD)
Author: Guoziying22 <guoziying22@mails.ucas.ac.cn>
Date: Tue Mar 4 18:27:48 2025 +0800

    add icache
```

- 也可用带HEAD的表达式来代替<commit hash>
 - HEAD表示当前版本，HEAD^表示上个版本，HEAD^^表示上上个版本
 - 向上n个版本写作HEAD~n

注：git reset命令会导致回退路径上所有修改移入暂存区，--hard选项可直接抛弃这些修改，慎用--hard选项

Git 用法：对本地仓库中的修改不满意，要回退（4）

❑ 回退后继续开发，可能在推送远程仓库（git push命令）时出现代码冲突

- 经常有同学搞不懂里面的关系，导致提交或推送出错

❑ 有一种比较笨（但或许更稳妥）的办法

- 从GitLab网站上找到想要回退到的旧版本，复制文件或直接下载，将其覆盖到工作区，然后**作为新的版本**进行 add、commit

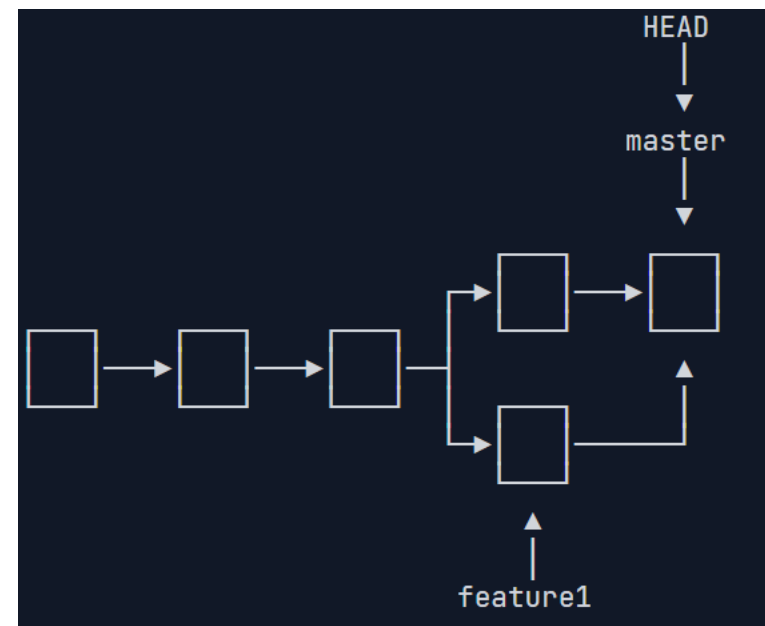
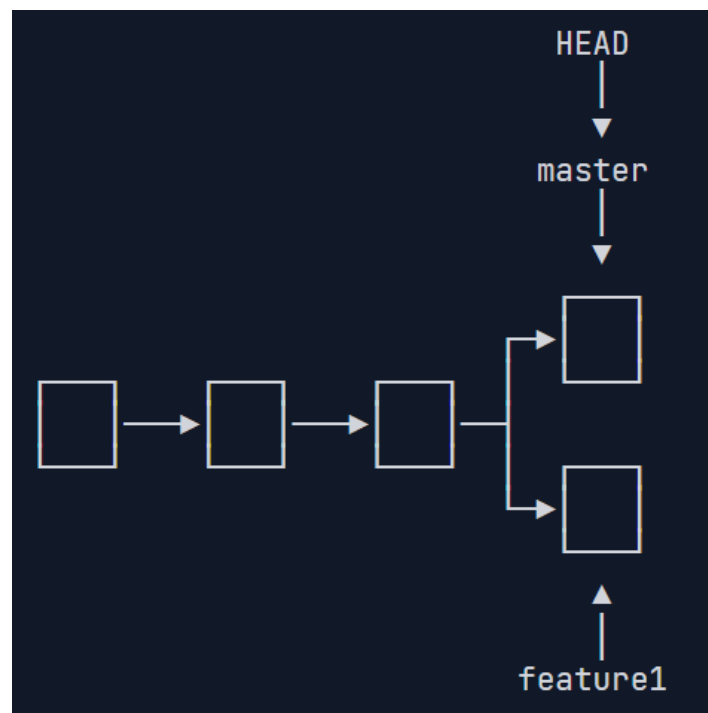
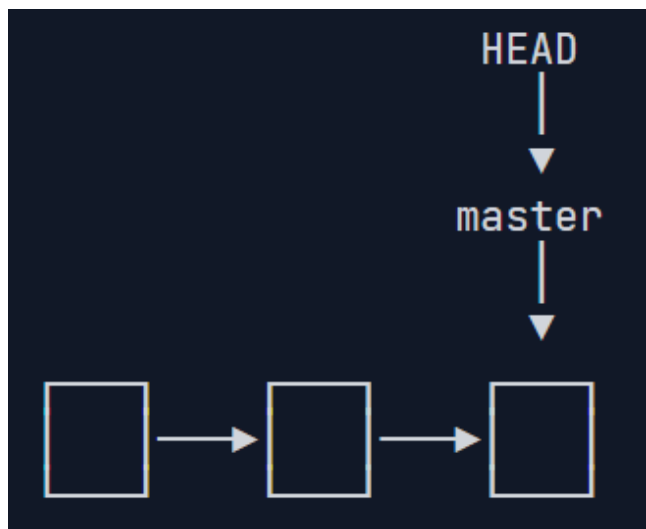
❑ 还有一种更好的开发方法

- 在开始某一功能开发或回退修改时，都生成不同的分支（branch）
- 当分支功能稳定（Fixed）后，再合并到实验项目默认的工作分支（master）上

Git 用法：分支 (branch)



- 分支：在更大的粒度上管理代码，使用分支意味着你可以从开发主线上分离开来，然后在不影响主线工作的同时继续工作。



Git 用法：分支 (branch)



□ 分支：在更大的粒度上管理代码，允许从主线分离，不影响主线开发。

□ 常用命令：

- `git branch` //查看分支
- `git branch <branch-name>` //创建分支
- `git checkout <branch-name>` //跳转到某分支，加-b就是创建并跳转
- `git branch -D <branch-name>` //（强制）删除分支
- `git merge` //合并分支

□ 操作相对复杂但管理效果好，感兴趣的同学可自学使用

- 可在菜鸟教程（<https://www.runoob.com/git/git-tutorial.html>）中搜索“Git分支管理”或查阅互联网上其它资料

Git 用法：忽略文件



- ❑ Git仓库初始化时，默认跟踪同步该目录下的所有文件
- ❑ 有些文件，例如源代码编译生成的二进制文件，同步它们不仅没有用还会造成很大的开销（主要是本地和云端的存储开销）
- ❑ 可以通过在Git仓库目录里按一定规则添加编写忽略文件 `.gitignore` 来指定哪些文件应被git忽略，也就是不要参与同步
- ❑ 在本实验中老师应该已经写好，如果今后大家用的时候可上网查找教程，这里就不详细介绍了

```
.gitignore
*.swp
5 *.npy*
6 result*.txt
7 *.mat
8 *.jpg
9 output
10 *.swp
11 *.pyc
12 *.zip
13 *.pth
```

- ❑ 如果需要学习了解更多Git的有关知识，可直接上网搜索相应内容，互联网上有丰富的资料足以满足我们的需要。例如一些比较全面的教程：
- ❑ 助教编写讲义：<http://8.152.206.202/ucas-cod-2021-dev/gituserguide>
- ❑ 菜鸟教程：<https://www.runoob.com/git/git-tutorial.html>
- ❑ 知乎文章：<https://zhuanlan.zhihu.com/p/30044692>
- ❑ CSDN文章：<https://blog.csdn.net/zhzijun/article/details/115727814>
- ❑ 参阅在线手册：
 - 英文文档：<https://git-scm.com/docs>
 - 中文手册：<https://git-scm.com/book/zh/v2>

問 与 答



中国科学院大学
University of Chinese Academy of Sciences



中科院计算所
INSTITUTE OF COMPUTING TECHNOLOGY, CAS